

9

POLYMORPHISM



This chapter is about the second major tenet of OOP: *polymorphism*. Its component parts are from the Greek: the prefix *poly* means “much” or “many,” and *morphism* means “shape,” “form,” or “structure.”

So, *polymorphism* essentially means *many forms*. I’m not talking about a *Star Trek*–style shape-shifting alien—in fact, it’s quite the opposite. Rather than one thing taking on many shapes, polymorphism in OOP is about how multiple classes can have methods with the exact same names. This will eventually give us a highly intuitive way to act on a collection of objects, independent of what class each came from.

OOP programmers often use the term “send a message” when we talk about client code calling a method of an object. What the object should do when it receives the message is up to the object. With polymorphism, we can send the same message to multiple objects, and each object will react differently depending on what it’s designed to do and the data available to it.

In this chapter, I’ll discuss how this ability allows you to build packages of classes that are easily extensible and predictable. We’ll also use polymorphism with operators to make the same operators perform different operations depending on the data types they are working with. Finally, I’ll